Phase 1 — Technical Research Notebook

Project: *The Algorithmic Panopticon: How AI Amplifies Security Exploits in Online Communities*
Phase: 1 — Technical & Systems Analysis **Owner / Team:** Mohammad Wael (and team) **Date Created:** 2025-10-18

Table of Contents

---

1. Scope & Research Questions

Scope (short): Discord (primary), public client-side mods (e.g., Vencord), ephemeral messaging behaviors, local cache artifacts, and conceptual AI pipelines.

Primary Research Questions: - Which client-side plugin features explicitly or implicitly bypass platform security/privacy controls?
- What artifacts of “deleted” or “ephemeral” messaging persist on local systems or in transit?
- What data formats, identifiers, or metadata enable automated ingestion/correlation by an AI?
- What constraints limit large-scale automated exploitation today?
- What realistic AI pipelines (inputs → processing → outputs) could scale these vulnerabilities?

---

2. Annotated Bibliography (public sources)

Add at least 8–10 entries here. For each entry use this mini-template:

- **Title / Source:**
  URL:
  Type: (e.g., GitHub repo, DFIR blog, Discord API docs, Reddit thread)
  Short note (3 bullets):

---

3. Plugin Audit Table (static analysis)

Instructions: Only perform **static** reviews of public repo code and docs. Record commit hashes or release tags when possible.

Plugin	Version / Commit	Feature	Hook(s) Observed	Stores Locally?	Data Types	Security Control Bypassed	Notes
MessageLogger (for Vencord)	“Temporarily logs deleted and edited messages.	Logs deleted & edited messages; allows	Hooks into message-delete / message-edit events (e.g., `onMessageDel`	Yes — plugin description says it saves logs to file or IndexedDB. (GitHub)	Text (deleted message content), message IDs, timestamps, channel IDs, user IDs, attachments. (GitHub)	Expected deletion of messages won’t remove them from the client’s log history	The plugin is open source; commit 6e7996… shows code changes in `src/plugins/messageLogger/index.tsx`

Plugin	Version / Commit	Feature	Hook(s) Observed	Stores Locally?	Data Types	Security Control Bypassed	Notes
	” (vencord.dev)	searching message history in mods. (GitHub)	`ete`) – see commit “messageLogger: fix ignore guild (#1632)” (git.derg.cz)				(git.derg.cz)
Discord Desktop Client Cache	— (version unspecified)	Persisting cached content after message/file deletion in official client. (Pen Test Partners)	Not a plugin hook per se – the official client’s cache system stores attachments and message data in a Chromium “Simple Cache” format. (Pen Test Partners)	Yes — as documented: `%AppData%\Discord\Cache\Cache_Data` and `data_#` / `f_*` files remain. (Pen Test Partners)	Cached attachments, image thumbnails, message fragments, webhook URLs, API call residues. (Pen Test Partners)	Deletion of a message in the UI does *not* guarantee removal from local cache. (LJMU Research Online)	This is outside of a mod – this demonstrates a platform-native persistence vulnerability
AlwaysTrust	(Plugin as listed on Vencord Plugins page) (vencord.dev)	Removes the “untrusted domain” & “suspicious file” pop-up warnings in the client.	Not explicitly documented which event hooks, but the code path shows patching of “suspicious file popup” methods. (git.catvibers.me)	N/A (not clearly stated)	Domain links, file trust prompts	User-interface / security warning bypass	Exists to override warnings about unsafe files/links.
Anonymise FileNames	Commit b3819228ed… in Vencord repo (Mar 2024) (git.derg.cz)	Anonymises uploaded file names (randomises or replaces them).	Upload file path modifications (uploadFiles:(...args)) in plugin code. (git.derg.cz)	N (it changes file name before upload)	Filenames, attachment metadata	Integrity/traceability of uploaded files; reduces forensic trace of source filename	Script modifies upload process; shows how metadata can be changed client-side.
MessageLogger	Public Vencord plugin by rushii, V, AutumnVN, etc. (vencord.dev)	Logs deleted and edited messages (“temporarily logs deleted and edited messages”).	Hooks onMessageDelete, onMessageEdit as indicated in community wiki. (wiki.vencord.dev)	Y — saves messages to JSON/IndexedDB according to GitHub repo. (GitHub)	Text content, message IDs, timestamps, attachments	The UI expectation that deleted message = gone is bypassed	Very clear case of client-side logging of deleted data for later retrieval.

Audit checklist (for each plugin) - Repo path(s) inspected:
- Manifest / metadata (author, license, version):
- Event hooks & API usage (list):
- Local storage methods (filesystem, DB, IndexedDB):
- Network calls (endpoints, hosts) — *only if visible in code*:
- Any obfuscation / minified bundles:

Plugin	Code path / hook detail	Storage / persistence mechanism
vc-message-logger-enhanced	In the GitHub repo *Syncxv/vc-message-logger-enhanced*, files such as LoggedMessageManager.ts and index.tsx handle deletion hooks. (GitHub) The index.tsx sets up event listeners for message	The README states that the plugin “saves messages to a json file” (in earlier versions) and later “moved from JSON to IndexedDB” in

Plugin	Code path / hook detail	Storage / persistence mechanism
	deletion and editing (e.g., onMessageDelete, onMessageEdit).	version 4.0.0. (GitHub) So local storage = IndexedDB (or JSON file fallback).
AlwaysTrust	In the Vencord plugin path src/plugins/alwaysTrust.ts (version v1.2.1) the code patches client methods: e.g., it finds .displayName="MaskedLinkStore" then replaces .isTrustedDomain=function(){return true} effectively overriding the domain-trust check. (Catvibers Git)	No explicit data-logging storage; the plugin changes UI/logic at runtime — i.e., bypasses alerts/trust checks rather than storing user content.
AnonymiseFileNames	Code is in src/plugins/anonymiseFileNames/index.tsx within the Vencord repo; it hooks into the upload path (uploadFiles handler) and transforms filenames before they're sent. (commit b3819228…)	Doesn't store files locally — transforms metadata client-side pre-upload. So the storage mechanism is "network/metadata modification" rather than local store.
MessageLogger (the simpler/plugin listing)	On the Vencord Plugin page: "Temporarily logs deleted and edited messages." (vencord.dev) The code path corresponds to the plugin's event hooks for onMessageDelete and onMessageEdit.	Similar storage behavior as enhanced version: presumably local logs (JSON/IndexedDB) of deleted/edited messages.
Discord Desktop Client Cache Artifact	For the official client: cached files under %AppData%\Discord\Cache\Cache_Data, data_*, f_*. DFIR blog posts show these contain message fragments/attachments. (Reddit)	Local storage = client cache (browser/Chromium-style). Persisted after deletion unless manually cleared; not tied to a plugin.

---

4. Cache & Persistence Observations (sandbox)

Instructions: Use a private test server and dummy accounts. Do **not** enable or run third-party plugin binaries against other people's systems. Capture OS paths, filenames, timestamps.

Plugin	OS	Persistent method	Persisted After Deletion? (Y/N)	Recovery Method (public ref)	Notes
MessageLogger	All	In-memory cache only (Discord MessageCache)	**N**	Not recoverable - exists in RAM only	Data stored in Discord's native MessageCache; cleared on reload/restart. No disk persistence.
AlwaysTrust	All	Settings in localStorage/Vencord settings file	**Y**	Standard browser localStorage recovery or Vencord	Only stores plugin enabled/disabled state and config. No message/link data stored.

				settings backup	
AnonymiseFileNames	All	Settings in localStorage/Vencord settings file	**Y**	Standard browser localStorage recovery or Vencord settings backup	Only stores anonymization method settings (random/consistent/timestamp). Original filenames never logged.

Artifact Path	OS	Observed Content	Persisted After Deletion? (Y/N)	Recovery Method (public ref)	Notes
IndexedDB (VencordData)	Windows	`%AppData%\discord\IndexedDB\https_discord.com_0.indexeddb.leveldb\`	**Y**	IndexedDB forensic tools, leveldb parsers	Stores Vencord plugin settings, notification logs, custom data. Survives app restart.
IndexedDB (VencordData)	macOS	`~/Library/Application Support/discord/IndexedDB/https_discord.com_0.indexeddb.leveldb/`	**Y**	IndexedDB forensic tools, leveldb parsers	Same as Windows
IndexedDB (VencordData)	Linux	`~/.config/discord/IndexedDB/https_discord.com_0.indexeddb.leveldb/`	**Y**	IndexedDB forensic tools, leveldb parsers	Same as Windows
localStorage (Vencord_settingsDirty)	Windows	`%AppData%\discord\Local Storage\leveldb\`	**Y**	LevelDB parsing tools	Contains Vencord settings and plugin configurations

Artifact Path	OS	Observed Content	Persisted After Deletion? (Y/N)	Recovery Method (public ref)	Notes
localStorage (Vencord_settingsDirty)	macOS	`~/Library/Application Support/discord/Local Storage/leveldb/`	**Y**	LevelDB parsing tools	Same as Windows
localStorage (Vencord_settingsDirty)	Linux	`~/.config/discord/Local Storage/leveldb/`	**Y**	LevelDB parsing tools	Same as Windows
Discord Message Cache	All	In-memory only (Electron process RAM)	**N**	Memory dump analysis (volatile)	Discord's native MessageCache; MessageLogger piggybacks on this. Lost on process exit.

5. AI Exploitation Model Templates

For each conceptual model, complete the template below and draw a simple ASCII or Mermaid diagram if helpful.

Model: Recon Agent (example)

Inputs: deleted message logs (text), message metadata (timestamps, msgIDs), user metadata (display names) — *all anonymized in research outputs*.

Processing Steps (high level): 1. Preprocessing: normalize text, remove stopwords, mask obvious PII. 2. Embedding & clustering: semantic embeddings → cluster by topic/personality indicators. 3. Behavioral inference: apply psycholinguistic models to infer traits. 4. Output: persona summaries, high-value target lists.

Outputs: persona profiles (non-PII), risk scores, recommended social-engineering vectors (***do not*** implement or provide step-by-step attack instructions).

Feasibility: High/Medium/Low — justify with references.

Ethical/Harms: Describe potential harms and mitigation strategies.

Mermaid diagram suggestion:

```
flowchart LR
  A[Deleted logs] --> B[Preprocess]
```

```
B --> C[Embeddings & Clustering]
C --> D[Behavioral Inference]
D --> E[Persona Profiles]
```

---

6. Findings Synthesis (2–3 page summary)

Use this section to draft the technical narrative you will hand off to Phase 2. Structure suggestions: - Executive summary (1 paragraph) - Key technical findings (bulleted) - What assumptions users make vs. reality - Potential AI amplification vectors (high-level, non-actionable) - Limitations & open questions

---

7. IRB Notes & Ethics Checklist

- Work restricted to public repos and controlled sandbox.
- No collection of real user data.
- All forum content paraphrased and anonymized.
- All outputs will be non-actionable and high-level.

IRB draft bullet points: - Study aims and non-malicious intent.
- Data sources (public code, sandbox artifacts, paraphrased public forum posts).
- Privacy & de-identification steps.
- Risk mitigation and storage policies.

---

8. To-Do / Action Items (editable checklist)

☐ Draft 2–3 page technical summary for Phase 2.